NYRA Platform - Complete System Documentation

> **A comprehensive guide to understanding every system in the NYRA platform**
>
> This document explains how everything works in simple terms, from user interactions to AI agents, from data storage to real-time communication.

📚 Table of Contents

1. Memory System
2. Supabase Database
3. Agentic System
4. Authentication & Security
5. Billing & Credits
6. Knowledge Base
7. Triggers & Automation
8. Integration System (MCP)
9. Sandbox Environment
10. Tools System
11. File Management
12. Real-time Communication
13. Worker & Task Queue
14. API Architecture
15. Frontend Architecture
16. Mobile App
17. Admin System
18. Analytics & Tracking
19. Notifications
20. Deployment & Infrastructure

---

1. Memory System

What It Does

The AI remembers information about you across conversations, making interactions more personalized.

How It Works

Step 1: Conversation Happens

You chat with the AI agent about your preferences, projects, or personal details.

Step 2: Background Analysis

After the conversation ends, a background worker analyzes the chat:

- An AI model reads through the messages
- It decides if there's anything worth remembering
- Skips small talk, remembers important facts

Step 3: Extraction

The AI extracts specific memories:

- **Facts**: Your name, job, company, location
- **Preferences**: Tools you like, communication style
- **Context**: What you're building, your tech stack
- **Summaries**: Key decisions or insights

Step 4: Storing

Each memory is:

- Converted to a vector (mathematical representation)
- Stored in database with metadata
- Tagged by type and confidence score

Step 5: Retrieval

When you start a new conversation:

- Your query is converted to a vector
- Database finds similar memories
- Most relevant ones are added to AI context
- AI uses them to personalize responses

Key Concepts

- **Automatic**: No manual effort required
- **Smart**: AI decides what's important
- **Semantic**: Finds memories by meaning, not exact words
- **Privacy**: You can view/delete all memories
- **Tiered**: Higher plans get more memory storage

---

2. Supabase Database

What It Is

Supabase is your data storage and backend service - think of it as the brain that remembers everything.

What It Provides

- **Database**: PostgreSQL (stores all your data)
- **Authentication**: User login and security
- **Storage**: File uploads (avatars, documents)
- **Real-time**: Live updates across devices
- **API**: Automatic endpoints for data access

How Data Is Organized

Main Data Collections

User & Accounts

- User profiles and authentication
- Account settings and preferences
- Team memberships

Conversations

- Projects (workspaces for organizing work)
- Threads (individual conversations)
- Messages (the actual chat)
- Agent runs (when AI executes tasks)

AI Agents

- Agent configurations (personality, tools)
- Knowledge base files
- Triggers (automation rules)

Billing

- Subscriptions (your plan)
- Credits (usage currency)
- Payment history

Features

- Memories (AI memory storage)
- Notifications

- User presence (online/offline)
- Analytics data

How It Works

Frontend → Supabase

1. User logs in via website/app
2. Gets a JWT token (like a key card)
3. Uses token for all requests
4. Can only access their own data (security)

Backend → Supabase

1. Backend has master key (full access)
2. Performs complex operations
3. Creates/updates data on user's behalf
4. Handles background tasks

Security Model

- **Row-Level Security (RLS)**: Users can only see their data
- **JWT Authentication**: Secure token-based login
- **Service Key**: Backend has elevated permissions
- **Encryption**: Sensitive data is encrypted

Why Supabase?

- Managed database (no server maintenance)
- Built-in authentication
- Real-time updates out of the box
- Vector search for AI features
- Scales automatically

---

3. Agentic System

What It Is

The core AI system that powers autonomous agents - they can think, plan, and execute tasks.

Architecture

Components

1. Thread Manager

- Manages conversation state
- Keeps track of messages
- Coordinates tool execution

2. Agent Configuration

- System prompt (personality)
- Available tools
- Integrations (external services)
- Knowledge base access

3. Tool System

- 30+ built-in tools
- External integrations via MCP
- Parallel execution capability

4. Execution Engine

- Sends messages to AI models
- Handles tool calls
- Manages context window
- Streams responses

How An Agent Runs

Step 1: User Sends Message

You type a message in the chat.

Step 2: Context Building

System gathers context:

- Previous messages in thread
- Relevant memories about you
- Agent's system prompt
- Available tools and their descriptions
- Knowledge base information

Step 3: AI Processing

Message sent to AI model (GPT-4, Claude, etc.):

- AI reads context and your message
- Decides on response strategy
- May call tools to gather information
- May execute actions (create files, search web, etc.)

Step 4: Tool Execution

If AI needs tools:

- Calls are executed (parallel if possible)
- Results returned to AI
- AI processes results
- Continues or responds

Step 5: Response

Final response streamed back to you in real-time.

Agent Types

Basic Agents

- Simple chat assistants
- Limited tools
- Good for Q&A

Custom Agents

- User-configured personality
- Selected tools
- Custom knowledge base
- Integrations

Specialized Agents

- Pre-built for specific tasks
- Email management
- Lead generation
- Research
- Content creation

How Tools Work

Tool Definition

Each tool has:

- Name and description

- Input parameters
- Output format
- Usage guidelines

Tool Execution

1. AI decides to use a tool
2. Provides parameters
3. Tool executes (in sandbox if needed)
4. Returns result
5. AI incorporates result into response

Tool Categories

- **File Operations**: Create, edit, read files
- **Web Tools**: Search, browse, scrape
- **Code Execution**: Run scripts, commands
- **Integrations**: Gmail, Slack, GitHub, etc.
- **AI Tools**: Vision, presentations, analysis

---

4. Authentication & Security

User Authentication Flow

Sign Up

1. User enters email and password
2. Supabase creates account
3. Verification email sent
4. User confirms email
5. Account activated

Login

1. User enters credentials
2. Supabase validates
3. Returns JWT token
4. Token stored (browser/app)
5. Token used for all requests

Session Management

- Token auto-refreshes before expiry

- Can sign out to invalidate
- Multi-device support
- Persistent sessions

Security Layers

Row-Level Security (RLS)

- Database-level protection
- Users only see their data
- Admin has elevated access
- Policies enforce rules automatically

API Security

- JWT validation on every request
- Rate limiting (prevents abuse)
- CORS protection (blocks unauthorized origins)
- Input validation

Data Encryption

- Passwords hashed (bcrypt)
- Sensitive data encrypted at rest
- HTTPS for all communications
- Secrets stored securely

Permission System

User Roles

- **User**: Normal access
- **Admin**: Analytics and management
- **Super Admin**: Full system access

Resource Ownership

- Projects belong to accounts
- Threads belong to projects
- Agents belong to accounts
- Can't access other users' resources

---

5. Billing & Credits

How It Works

Credit System

Credits are the "currency" for using AI:

- 1 credit ≈ 1 API call to AI models
- Different models cost different credits
- Credits deducted on usage
- Balance tracked in real-time

Credit Types

Daily Credits

- Refresh every 24 hours
- All tiers get 100 daily credits
- Use-it-or-lose-it basis

Monthly Credits

- Included with subscription
- Roll over if unused (depends on tier)
- Refill on renewal

Purchased Credits

- Buy extra credit packages
- Never expire
- Used after daily/monthly credits

Subscription Tiers

Free Tier

- Daily credits only
- Limited features
- 10 chat limit
- Basic mode only

Paid Tiers (Plus, Pro, Ultra)

- Monthly credit allocation
- Custom agents
- Scheduled triggers
- App integrations

- Advanced AI mode
- More concurrent runs

Billing Flow

Subscription

1. User selects plan
2. Stripe processes payment
3. Webhook confirms payment
4. Credits granted automatically
5. Features unlocked

Credit Usage

1. User sends message
2. AI model called
3. Credits deducted atomically
4. Balance updated
5. User notified if low

Renewal

1. Subscription renews monthly/yearly
2. Stripe charges card
3. Webhook received
4. New credits granted
5. Cycle continues

Payment Providers

Stripe (Web)

- Credit cards
- Subscriptions
- One-time payments

RevenueCat (Mobile)

- App Store (iOS)
- Play Store (Android)
- Subscription management

---

6. Knowledge Base

What It Is

A storage system for files and documents that agents can search and reference.

Two Types

Global Knowledge Base

- Files you upload separately
- Organized in folders
- Can enable/disable per agent
- Searchable by all enabled agents

Local Knowledge Base (Sandbox)

- Files created during conversations
- Automatically indexed
- Available within the conversation
- Temporary (per session)

How It Works

Upload

1. User uploads document
2. File processed and parsed
3. Content extracted
4. Indexed for search

Indexing

- Text extracted from files
- Broken into chunks
- Each chunk gets embedding (vector)
- Stored for semantic search

Search

1. Agent needs information
2. Query converted to vector
3. Database finds similar content
4. Relevant chunks returned
5. Agent uses in response

Supported Files

- PDFs
- Word documents
- Text files
- Markdown
- Code files
- Spreadsheets

Why It's Useful

- Agents can reference your documents
- No need to copy-paste content
- Automatic updates when files change
- Semantic search (understands meaning)

---

7. Triggers & Automation

What Triggers Do

Automate agent execution without manual input.

Types of Triggers

Scheduled Triggers

Run agents at specific times:

- **Daily**: Every morning at 9 AM
- **Weekly**: Every Monday
- **Monthly**: First of the month
- **Custom**: Any cron schedule

Example: Daily email summary agent runs every 8 AM.

Event Triggers

Run when external events happen:

- New email received (Gmail)
- New Slack message
- Form submitted
- Calendar event
- GitHub push

Example: Agent responds to support tickets automatically.

How Triggers Work

Setup

1. User creates trigger
2. Specifies schedule or event
3. Selects agent to run
4. Defines prompt/instructions
5. Activates trigger

Execution

1. Trigger time arrives or event happens
2. System creates new agent run
3. Agent executes automatically
4. Results saved or sent
5. Next trigger scheduled

Monitoring

- View trigger history
- See success/failure
- Enable/disable anytime
- Edit trigger settings

Use Cases

- Daily reports
- Email automation
- Monitoring and alerts
- Data synchronization
- Scheduled research
- Regular backups

---

8. Integration System (MCP)

What MCP Is

MCP (Model Context Protocol) connects agents to external services like Gmail, Slack, GitHub, etc.

How It Works

Architecture

Think of MCP as a universal adapter:

- NYRA speaks one language
- MCP servers translate for each service
- 2700+ services available

Connection Flow

Step 1: Search User searches for integration (e.g., "Gmail").

Step 2: Create Profile Create credential profile for the service.

Step 3: Authenticate User authorizes NYRA to access their account.

Step 4: Select Tools Choose which actions agent can perform.

Step 5: Use Agent can now use the service in conversations.

Available Integrations

Communication

- Gmail, Outlook
- Slack, Teams, Discord
- SMS services

Productivity

- Notion, Asana, Trello
- Google Drive, Dropbox
- Calendar apps

Development

- GitHub, GitLab
- CI/CD platforms
- Cloud services

Business

- Salesforce, HubSpot
- Analytics platforms
- Payment processors

Security

- OAuth authentication

- Tokens encrypted
- Minimal permissions
- User controls access
- Can revoke anytime

---

9. Sandbox Environment

What It Is

A secure, isolated environment where agents execute code and tools.

Why Sandboxes?

- **Safety**: Code can't harm your computer
- **Isolation**: Each agent has its own space
- **Consistency**: Same environment every time
- **Security**: Can't access sensitive data

What's Inside

Operating System

- Debian Linux (lightweight)
- Python 3.11 pre-installed
- Common tools and libraries
- Network access (controlled)

File System

- `/workspace` directory for files
- Temporary storage
- Can install packages
- Files persist during conversation

Capabilities

- Run Python scripts
- Execute shell commands
- Install dependencies
- Create and edit files
- Browse the web
- Process images

How It Works

Creation

1. User starts conversation
2. System allocates sandbox
3. Environment prepared
4. Agent gains access

Execution

1. Agent decides to use tool
2. Command sent to sandbox
3. Executes in isolated environment
4. Results returned to agent
5. Agent continues

Cleanup

- Sandbox lives during conversation
- Destroyed after inactivity
- Files can be saved/downloaded
- Resources reclaimed

Powered By

Daytona - Provides the sandbox infrastructure:

- Container-based isolation
- Fast provisioning
- Automatic scaling
- Resource management

---

10. Tools System

What Tools Are

Functions agents can call to perform actions beyond text generation.

Categories

Core Tools (Built-in)

File Operations (`sb_files_tool`)

- Create, read, edit, delete files

- Natural language file editing
- Supports all text formats

Web Browser (`browser_tool`)

- Navigate websites
- Extract information
- Fill forms
- Take screenshots
- Automated web tasks

Shell Commands (`sb_shell_tool`)

- Execute terminal commands
- Run scripts
- System operations
- Install packages

Web Search (`web_search_tool`)

- Search the internet
- Gather information
- Research topics

Vision (`sb_vision_tool`)

- Analyze images
- OCR (text from images)
- Understand screenshots

Presentations (`sb_presentation_tool`)

- Create professional slideshows
- 18+ template styles
- Export to HTML/PDF

Knowledge Base (`sb_kb_tool`)

- Semantic file search
- Access knowledge base
- Query documents

Spreadsheets (`sb_spreadsheet_tool`)

- Create Excel files
- Formulas and formatting
- Data visualization

External Tools (via MCP)

- Service-specific actions
- Gmail: Send/read emails
- Slack: Post messages
- GitHub: Create issues
- Thousands more...

How Tools Are Defined

Each tool has:

- **Name**: Identifier
- **Description**: What it does
- **Parameters**: Required inputs
- **Usage Guide**: Instructions for AI
- **Schema**: Formal definition

Tool Execution

Sequential

Tools execute one after another:

- Predictable order
- Each depends on previous
- Slower but reliable

Parallel

Multiple tools execute simultaneously:

- Faster execution
- Independent operations
- Better for bulk operations

Tool Selection

- AI chooses tools based on task
- Can use multiple tools
- Chains tools together
- Returns results in context

---

11. File Management

File System Structure

Workspace

- Main directory: `/workspace`
- Organized by user/project
- Persists during session
- Can be backed up

File Types Supported

- Documents (txt, md, pdf, docx)
- Code (py, js, ts, html, css, etc.)
- Data (csv, json, xml)
- Images (png, jpg, svg)
- Presentations (html, pdf)

File Operations

Create

- Agent creates new file
- Chooses name and location
- Writes initial content
- Sets permissions

Read

- Agent accesses file content
- Can read specific sections
- Supports large files
- Binary file handling

Edit

- Natural language editing
- "Change line 10 to say X"
- Smart replacements
- Multiple edits at once

Delete

- Remove files/directories

- Safety confirmations
- Cleanup operations

File Sharing

- Download from chat
- Preview in browser (HTML files)
- Upload to cloud storage
- Share via links

Special Features

AI-Powered Editing

Instead of manual find-replace:

- "Add error handling to function X"
- "Refactor this class to use async"
- "Fix the bug on line 45"

Agent understands intent and makes changes.

Preview Server

HTML files auto-exposed on port 8080:

- Create website → instant preview
- No server setup needed
- Share live URL

---

12. Real-time Communication

What It Enables

Live updates across all devices without refreshing.

How It Works

WebSocket Connections

- Persistent connection between client and server
- Bidirectional communication
- Low latency
- Automatic reconnection

Supabase Real-time

- Database changes broadcast instantly
- Subscribe to specific tables/rows
- Filter by user/project
- Presence tracking

What's Real-time

Messages

- New messages appear instantly
- Typing indicators
- Delivery status
- Read receipts

Agent Status

- Running/completed/failed
- Progress updates
- Tool execution status

Presence

- Who's online/offline
- Last seen time
- Active now indicator

Notifications

- System notifications
- User mentions
- Task completion alerts

Implementation

Frontend

1. Opens WebSocket connection
2. Subscribes to relevant channels
3. Receives updates
4. Updates UI automatically

Backend

1. Database changes trigger events

2. Supabase broadcasts to subscribers
3. Filters by permissions
4. Delivers to connected clients

Benefits

- No polling needed
- Instant updates
- Battery efficient
- Synchronized across devices

---

13. Worker & Task Queue

What It Is

Background job system for heavy/async operations.

Architecture

Redis Streams

- Queue for background tasks
- Multiple consumers (workers)
- Persistent (survives restarts)
- Priority support

Worker Process

- Separate from API server
- Processes tasks asynchronously
- Scalable (add more workers)
- Handles failures gracefully

Task Types

Agent Runs

- Execute AI agent conversations
- Long-running operations
- Tool execution
- Result storage

Memory Extraction

- Process conversations for memories
- AI analysis
- Embedding generation
- Database storage

File Processing

- Parse uploaded documents
- Extract text
- Generate embeddings
- Index for search

Billing Operations

- Credit grants
- Renewal processing
- Usage tracking
- Invoice generation

Notifications

- Send emails
- Push notifications
- System alerts
- Scheduled messages

How It Works

Enqueue

1. API receives request
2. Creates task message
3. Pushes to Redis queue
4. Returns immediately

Process

1. Worker polls queue
2. Picks up task
3. Executes operation
4. Updates status
5. Cleans up

Monitor

- Task status tracking
- Retry on failure
- Dead letter queue
- Metrics collection

Benefits

- Non-blocking API
- Handles load spikes
- Fault tolerant
- Scalable architecture

---

14. API Architecture

Design

RESTful API

- Standard HTTP methods (GET, POST, PUT, DELETE)
- JSON request/response
- Versioned endpoints (/v1/)
- OpenAPI documentation

FastAPI Framework

Python web framework:

- Fast and modern
- Auto-generated docs
- Type validation
- Async support

Structure

Routers

Organized by feature:

- `/v1/agents` - Agent management
- `/v1/threads` - Conversations
- `/v1/billing` - Subscriptions
- `/v1/memory` - Memory system

- `/v1/triggers` - Automation

Middleware

Runs on every request:

- Authentication checking
- Rate limiting
- Request logging
- Error handling
- CORS management

Request Flow

1. **Client sends request**
2. **Middleware processes**
    - Validates auth token
    - Checks rate limits
    - Logs request
3. **Router handles**
    - Finds endpoint
    - Validates input
4. **Business logic executes**
    - Database queries
    - External API calls
    - Processing
5. **Response returned**
    - JSON formatted
    - Status code
    - Headers

Authentication

JWT Tokens

- Included in Authorization header
- Validated on each request
- Contains user ID
- Expires after time

API Keys

- Alternative to JWT
- For programmatic access
- User-generated
- Can be revoked

Error Handling

- Standard HTTP codes
- Detailed error messages
- Validation errors
- Retry guidance

---

15. Frontend Architecture

Technology Stack

Next.js

React framework:

- Server-side rendering
- Static generation
- File-based routing
- API routes

React

UI library:

- Component-based
- State management
- Hooks for logic
- Virtual DOM

TypeScript

Type-safe JavaScript:

- Compile-time errors
- Better IDE support
- Self-documenting
- Refactoring safety

Tailwind CSS

Utility-first styling:

- Responsive design
- Dark mode support
- Consistent spacing
- Fast development

Architecture

Pages

Route-based structure:

- `/dashboard` - Main interface
- `/agents` - Agent management
- `/settings` - User settings
- `/auth` - Authentication

Components

Reusable UI pieces:

- `ChatInterface` - Conversation UI
- `AgentCard` - Agent display
- `MessageBubble` - Chat message
- `Button`, `Input`, etc.

State Management

- React Context for global state
- TanStack Query for server data
- Local state for UI
- Zustand for complex state

API Layer

- Centralized API functions
- Type-safe requests
- Error handling
- Loading states

Data Flow

1. **User interaction**
2. **Component updates state**

3. **State change triggers re-render**
4. **API call if needed**
5. **Response updates state**
6. **UI reflects changes**

Key Features

Optimistic Updates

- UI updates immediately
- Reverts if API fails
- Better user experience

Caching

- TanStack Query caches responses
- Reduces API calls
- Stale-while-revalidate pattern

Real-time

- WebSocket integration
- Live updates
- Presence tracking

---

16. Mobile App

Technology

React Native

- JavaScript for mobile
- Native performance
- Single codebase
- iOS and Android

Expo

Development platform:

- Build tools
- OTA updates
- Native modules
- Development client

Architecture

Similar to web but mobile-optimized:

- Native navigation
- Touch gestures
- Push notifications
- Offline support
- Biometric auth

Key Differences from Web

Navigation

- Stack-based navigation
- Drawer for menu
- Tab navigation
- Native transitions

Storage

- AsyncStorage for persistence
- Secure storage for tokens
- SQLite for offline data

Notifications

- Push notifications
- Local notifications
- Background sync

Platform APIs

- Camera access
- File picker
- Share sheet
- Haptic feedback

Development

Build Process

1. Code in TypeScript/React Native
2. Expo bundles JavaScript
3. Builds native apps
4. Distributes via stores

Updates

- Over-the-air (OTA) updates
- No app store approval needed
- Instant bug fixes
- Feature rollouts

---

17. Admin System

Purpose

Tools for platform management and monitoring.

Features

User Management

- View all users
- Search by email/ID
- Manage subscriptions
- Grant credits manually
- View user activity

Analytics Dashboard

- Signup metrics
- Subscription trends
- Revenue tracking
- User retention
- Feature usage

Billing Operations

- Process refunds
- Adjust subscriptions
- View payment history
- Handle disputes
- Credit management

Feedback Review

- User feedback submissions
- Categorization

- Response tracking
- Feature requests

Notification Management

- Send system announcements
- User notifications
- Email campaigns
- Push notifications

Access Control

Admin Role

- View analytics
- Read-only access
- Basic management

Super Admin

- Full system access
- Modify any data
- Billing operations
- User management

Security

- Separate authentication
- Audit logging
- Action confirmation
- Rate limiting

---

18. Analytics & Tracking

What's Tracked

User Metrics

- Signups (daily/weekly/monthly)
- Active users
- Retention rates
- Churn analysis

Usage Metrics

- Messages sent
- Agent runs
- Tool usage
- Credit consumption
- Feature adoption

Business Metrics

- Revenue (MRR, ARR)
- Subscription conversions
- Plan distribution
- Payment success rates
- LTV (Lifetime Value)

Implementation

Google Analytics

- Page views
- User flows
- Event tracking
- Conversion funnels

Vercel Analytics

- Web vitals
- Performance metrics
- Error rates
- Geographic distribution

Custom Analytics

- Database queries
- Aggregated metrics
- Historical trends
- Forecasting

Data Pipeline

1. **Events occur** (user action, system event)
2. **Tracked in database** (timestamped records)
3. **Aggregated periodically** (hourly/daily)

4. **Stored in analytics tables**
5. **Visualized in dashboards**

Privacy

- Anonymized where possible
- GDPR compliant
- User opt-out
- Data retention limits

---

19. Notifications

Types

In-App Notifications

- System messages
- Feature announcements
- Action required
- Success confirmations

Email Notifications

- Account updates
- Billing alerts
- Security notifications
- Marketing (opt-in)

Push Notifications (Mobile)

- Message received
- Agent completed
- Low credits
- Trigger executed

Delivery System

Creation

1. Event triggers notification
2. Message composed
3. Recipients determined
4. Delivery method chosen

Storage

- Stored in database
- Marked as read/unread
- Expiration dates
- Priority levels

Delivery

- Real-time via WebSocket
- Email via SendGrid
- Push via FCM/APNs
- Batched for efficiency

User Preferences

- Notification settings
- Channel preferences
- Frequency control
- Mute options

---

20. Deployment & Infrastructure

Architecture

Services

Frontend (Vercel)

- Next.js deployment
- CDN distribution
- Edge functions
- Automatic scaling

Backend (Cloud - AWS/GCP)

- FastAPI application
- Multiple instances
- Load balancing
- Auto-scaling

Database (Supabase)

- Managed PostgreSQL
- Automatic backups

- Connection pooling
- Read replicas

Queue (Redis)

- Managed Redis
- Persistence enabled
- High availability
- Cluster mode

Workers

- Background processing
- Multiple instances
- Task consumption
- Health monitoring

Sandbox (Daytona)

- Isolated environments
- Container orchestration
- Resource limits
- Auto-cleanup

Environments

Local

- Development on laptop
- Docker Compose
- Local Supabase
- Fast iteration

Staging

- Pre-production testing
- Separate database
- Test payments
- QA environment

Production

- Live user traffic
- High availability
- Monitoring
- Backup systems

Deployment Process

1. **Code push to GitHub**
2. **CI/CD pipeline runs**
    - Tests execute
    - Build creates artifacts
3. **Deploy to staging**
    - Smoke tests
    - QA approval
4. **Deploy to production**
    - Gradual rollout
    - Health checks
    - Rollback ready

Monitoring

Application Monitoring

- Error tracking (Sentry)
- Performance metrics
- API latency
- Database queries

Infrastructure Monitoring

- Server health
- Resource usage
- Network metrics
- Cost tracking

Alerts

- Critical errors
- Service downtime
- Performance degradation
- Budget exceeded

Backup & Recovery

Database Backups

- Daily automatic backups

- Point-in-time recovery
- Retention policy
- Tested restore process

Disaster Recovery

- Multi-region deployment
- Failover systems
- Data replication
- Recovery procedures

🎯 How Everything Connects

User Journey Example

1. **User logs in** → Authentication system validates
2. **Sees dashboard** → Frontend loads from Next.js
3. **Starts conversation** → Thread created in Supabase
4. **Sends message** → Saved to database
5. **Agent runs** → Task queued in Redis
6. **Worker processes** → Pulls from queue
7. **Sandbox allocated** → Daytona provisions environment
8. **Tools execute** → Actions performed in sandbox
9. **AI responds** → Streamed to frontend via WebSocket
10. **Memory extracted** → Background task analyzes conversation
11. **Credits deducted** → Billing system updates balance
12. **Notification sent** → User informed of completion

Data Flow

```
User Input → Frontend → API → Database
                ↓
            Task Queue → Worker → Sandbox → Tools
                ↓
            AI Model → Response → Frontend
                ↓
            Real-time → All Devices
```

📑 Key Principles

Separation of Concerns

Each system handles one responsibility clearly.

Asynchronous Processing

Heavy tasks run in background, API stays fast.

Security First

Authentication, encryption, and isolation everywhere.

Scalable Architecture

Add more servers/workers as load increases.

User Privacy

Data isolation, encryption, user controls.

Developer Experience

Clear APIs, good documentation, easy debugging.

🚀 Building Something Similar

What You Need

1. **Database Layer**: Supabase or PostgreSQL
2. **Backend API**: FastAPI, Express, or similar
3. **Frontend**: React/Next.js or Vue
4. **Queue System**: Redis, RabbitMQ, or SQS
5. **AI Integration**: OpenAI, Anthropic APIs
6. **Sandbox**: Docker, Kubernetes, or Daytona
7. **Authentication**: Supabase Auth, Auth0, or custom JWT
8. **Payments**: Stripe, RevenueCat
9. **Hosting**: Vercel (frontend), AWS/GCP (backend)

Core Components to Build

1. **User management** (signup, login, profiles)
2. **Conversation system** (threads, messages)
3. **AI integration** (model API calls)

4. **Tool system** (extensible functions)
5. **Billing** (subscriptions, credits)
6. **Admin panel** (user management, analytics)

Development Approach

1. Start with MVP (basic chat + AI)
2. Add authentication
3. Implement billing
4. Build tool system
5. Add automation (triggers)
6. Create integrations
7. Polish UI/UX
8. Scale infrastructure

This documentation provides a complete understanding of how NYRA works. Every system, every flow, every decision explained in simple terms. Use it as a reference to understand the platform or build something similar!